Extended Abstract

Title: Auditable AI Agents: An Immutable On-Chain Audit Trail for LLM Agent Actions
Author: Manikandan Sankaranarayanan, Department of Computer Science and Engineering, S Veersamy Chettiya College of Engineering (smak07.s@gmail.com)
Status: Draft, 2026-10-30. Results reported are from automated tests and first runs, not the final evaluation.

Background and problem

LLM agents increasingly act on their own: they read files, call APIs and make decisions. When something goes wrong, the question is what the agent actually did. Today the answer usually sits in a log file or database controlled by whoever runs the agent. Whoever controls it can alter or delete entries, and nobody else can prove the record was left untouched.

Approach

The system makes agent records tamper-evident in three layers.

1. **Hash-chained log.** Every agent step (task, model response, tool call, tool result or error, final answer) becomes an entry containing the SHA-256 hash of the previous entry. Editing, deleting, inserting or reordering an entry breaks the chain.
2. **Merkle batches.** Every B entries, their hashes are folded into one Merkle root. Domain-separated leaf and node hashing, and carrying up an unpaired node instead of duplicating it, prevent known Merkle-tree weaknesses.
3. **Blockchain anchor.** The root is written to a smart contract, which stores it with its entry range and block time. Only hashes are on-chain; the full log stays in ordinary storage.

A verifier reloads the stored log, rechecks the chain and each batch root, and reports the first altered entry, or the altered batch when the log was rewritten consistently.

Implementation

Python (pure functions and immutable data for the cryptographic logic), Solidity 0.8.28 for the contract, Hardhat for a local Ethereum chain and contract tests, and web3.py to connect them. The agent has three tools and can be driven by a scripted stand-in model, a local open-weight model (qwen3:8b through Ollama, no API key), or Claude through an adapter. All models share one signature, so the audit layer is model-independent.

Results to date

- Seven classes of tampering (payload, timestamp and stored-hash edits, deletion, insertion, reordering, truncation) were each applied to 300 random logs; **all 2,100 were detected**, and **none of 300 untouched**

logs was flagged.

- A log whose entries were rewritten with every later hash recomputed passes the hash chain but **fails against the on-chain roots**, which is the added value of anchoring.
- **Python and Solidity agree**: Merkle proofs computed in Python for trees of 1 to 100 entries verify inside the contract.
- A real run with a local 8-billion-parameter model produced a 12-entry log that was anchored in three batches and verified; anchoring cost about 79,500 to 93,100 gas per batch on the development chain.
- The test suite has 79 Python tests and 18 contract tests.

Limits

The chain proves a record was not changed after it was anchored. It does not prove the record was true, and entries written after the latest anchor are not yet protected. A compromised logger could write false entries before anchoring. After a consistent rewrite, only the affected batch, not the exact entry, can be located. The prototype runs on a local chain and has not been audited.

Remaining work

Measure gas against batch size, logging and anchoring overhead per agent step, and verification time and size from 100 to 100,000 entries; add a dashboard; consider signed entries; and complete the literature review around Crosby and Wallach's tamper-evident logging.

Keywords

LLM agents; audit trail; tamper-evident logging; Merkle tree; blockchain anchoring; smart contracts.